

AutoRTS 程序优化培训

主讲人：何健乐

01

原理与架构

复杂 = 简单模块的组合；先懂帧同步架构，再谈容器

- 第一性原理：简单模块 × 增删改查
- 架构：逻辑层 (BattleCore) 与表现层
- 帧同步的确定性 → 容器与排序纪律

开篇：第一性原理

复杂的东西，底层永远是简单的

**“The program is composed of simple, straightforward modules,
with each one doing one thing well.”**

—— Rob Pike 《Notes on Programming in C》

简单模块

复杂系统 = 简单模块的组合。容器 = 内存布局 + 增删改查；算法 = 查找 / 排序 / 遍历

业务本质

就像后端都是增删改查（CRUD），大部分游戏业务代码，本质上也是对数据的增删改查

锚定原理

写代码时自检：这段代码在做哪种操作？
容器选对了吗？顺序依赖了吗？

本课所有内容，都是这三个卡片的具体展开

AutoRTS 架构：逻辑层与表现层

先懂架构，再写代码

逻辑层 BattleCore (纯 C++)

帧同步 (Lockstep)：输入同步，结果逐帧一致

客户端 / 服务端双端复用同一份代码

确定性优先：定点数 FF24 / FFDec，不用 float

禁止调用表现层：不能碰 UI、渲染、Actor

表现层 BattleVisualization / UI

渲染、特效、音效、输入、界面

可自由使用引擎特性与各种容器

单向依赖逻辑层：读取状态，渲染表现

性能优先：每帧更新、缓存、节流

核心约束：逻辑层代码必须脱离表现层独立运行

帧同步的确定性，决定逻辑层的容器纪律

逻辑层的容器问题 = 正确性问题，不只是性能问题

风险点	为什么危险	纪律
排序不稳定	Sort 是快排，并列元素顺序不定	需要稳定时用 StableSort；比较器必须是严格全序
浮点运算	不同平台/指令集结果可能不同	用定点数（项目已用 FF24 / FFDec）

表现层则没有这些约束：UI / 特效 / Actor 怎么快怎么来

- 写逻辑层代码前，先问一句：这段代码在两端跑，结果会一样吗？

02

容器与算法

从内存布局看懂四种容器，选型即有答案

- 心智模型：TArray / TMap / TSet / TSortedMap
- TArray：新增 · 删除 · 查找
- TMap / TSet：Key 选型与查找姿势
- 字符串 · Algo 算法 · 决策表

容器的本质是内存布局

四个容器，一张表记住

容器	内存布局	查找	增删	迭代
TArray	连续内存	$O(n)$ 线性	尾部 $O(1)$ ，中间 $O(n)$	有序 · 缓存友好
TMap	哈希桶	$O(1)$ 按键	$O(1)$ ，可能 rehash	无序
TSet	哈希桶（只有键）	$O(1)$	$O(1)$	无序
TSortedMap	红黑树	$O(\log n)$	$O(\log n)$	有序

一句话：TArray 遍历最快；哈希容器按键最快；迭代顺序只信 TArray 和 TSortedMap

- 数据量小时，TArray 线性查找常常比哈希还快（缓存友好）
- 所以选型先看两个问题：遍历还是按键查？顺序要不要？

一句话选型

需求定，容器就定了

要遍历 · 要保序 · 数量小

单位组、武器槽、特效列表

TArray

按键取东西 · 查重

技能表、配置表、选中集合

TMap / TSet

有序遍历 + 按键查 · 读多写少

排行榜、时间序事件表

TSortedMap

先进先出

命令队列

TQueue

口诀：先问「遍历 or 按键查 or 有序」

TArray：新增元素

批量构造是扩容的高发区

```
// 不预留：每 Add 一次都可能扩容、搬运整个数组
Caches.Add(EffectCache);           // 还有一次拷贝构造

// 预留 + 原地构造：零拷贝、不扩容
Caches.Reserve(ExpectedCount);
Caches.Emplace(EffectID, ...);
```

Emplace：在容器内存里直接构造

Reserve：先声明「我会加 N 个」

循环批量塞数据的代码，开头先 Reserve

项目实测

- Emplace 207 处，意识良好；Reserve 仅 29 处，批量场景记得补

TArray：删除元素

删除前先问：这个数组的顺序重要吗？

API	保序	复杂度
Remove(Value)	是	O(n)
RemoveAt(Index)	是	O(n)
RemoveAtSwap(Index)	否	O(1)
RemoveAll(Pred)	批量	O(n) 一次遍历

```
// 单位阵亡移除：  
// 顺序无所谓 → O(1)  
Units.RemoveAtSwap(Idx);
```

遍历中删除的正确姿势

```
for (int32 i = Arr.Num() - 1; i >= 0; --i) // 倒序删，索引不乱  
if (ShouldRemove(Arr[i])) Arr.RemoveAtSwap(i);  
  
// 或一行：Arr.RemoveAll(谓词) — 项目已在大量使用
```

- RemoveAtSwap 全库仅 5 处，战斗内高频删除应多用

TArray: 查找是 $O(n)$

下结论前，先确认容器类型

```
// 先确认类型：SoundList 是 TMap<EWeaponSoundState, USoundBase*>
// (WeaponConfig.h:60) → 枚举 Key 哈希,  $O(1)$ , 没问题
if (!WeaponConfig->SoundList.Contains(State)) return;

// 反例：UnitEffectCaches 是 TArray, 手写线性查找  $O(n)$ 
// (BaseUnitActor.cpp:2040) → 换 TMap 或 Algo::FindByPredicate
for (auto& It : UnitEffectCaches)
    if (EffectKind == It.EffectID) return;
```

教学案例

看到 Contains 先别急着优化：查头文件确认是 TArray 还是 TMap、调用是每帧还是事件。

规则

- TArray 的 Contains / Find 每次都是全数组扫描
- 高频按键查找 → 换 TMap / TSet, 或按 ID 建索引

TMap / TSet: Key 的选型是第一优先级

哈希容器的开销 = 哈希计算 + 键比较

整数 / 枚举 ▶ 最快
FName ▶ 比较 $O(1)$, 全局表索引
FString ▶ 最贵: 长度哈希 + 逐字符比较

项目 24 处 TMap<FString>:

多数在 UI / SDK / 存档等低频路径, 可接受;

战斗内触达的 (RTSAssetManager.h:46 等) 建议换 FName / 整数 ID。

正面教材

- WeaponConfig.h:60 用枚举做 Key (EWeaponSoundState) $\rightarrow O(1)$
- RTSAssetSkinEffect.h:212 用 int 做 Key (AbilityEmitBombEffects) $\rightarrow O(1)$
- 新代码规则: 整数 / 枚举 > FName > FString; FString 只用于外部数据对接

TMap: 一次查找, 不要两次

Contains + [] 是两次哈希; Find 一次搞定

```
// 不推荐: Contains 一次哈希, [] 又一次
if (Map.Contains(Key) && Map[Key].bReady) { ... }

// 推荐: Find 返回指针, 判空即存在
if (const FWeaponData* Data = Map.Find(Key))
{
    if (Data->bReady) { ... }
}
```

FindOrAdd 的陷阱

找不到会默认构造并插入 —— 只读场景别用;

逻辑不变量用 FindChecked (找不到直接断言失败)。

- TMap 迭代是无序的 —— 需要有序输出时先排序 (见后页)
- 遍历 TMap 不要用 [] 取值 (每次又哈希一遍)

TSet: 去重与集合运算

TSet = 只有键的 TMap, Contains 是 O(1)

```
TSet<FUnit*> Picked; // 去重 / 查重 O(1)
if (!Picked.Contains(Unit)) Picked.Add(Unit);

A.Intersect(B); // 交集 — 项目已用 51 处
A.Union(B);     // 并集
A.Difference(B); // 差集
```

项目案例

SkinEffectManager.h:31 的 BuffStack 用 TSet<int>, 每帧 Contains O(1), 正确;

同文件另一类却用了 TArray<int> 同语义 —— 改代码时别把 TSet 改回 TArray。

- 「选中集合」「可交互集合」「互斥集合」优先 TSet
- 既要遍历又要查重: TArray (遍历) + TSet (查重) 双结构

字符串： FString / FName / FText

别用 FString 当 Key，别在热路径拼字符串

类型	内存	比较	用途
FString	堆分配 · 可变	逐字符 O(n)	显示、拼接、日志
FName	全局表索引 · 不可变	O(1)	ID、Key、Socket 名
FText	本地化包装	慢	面向玩家的文案

项目反模式：运行时拼字符串再转 FName

```
                // BaseUnitActor.cpp:2359 / Soldier.cpp:305 — 战斗内创建乘员时调用
return FName(*FString::Format(TEXT("Transport_{0}"), { SeatIndex + 1 }));
// 每次堆分配 + 查全局表；应预生成 TArray<FName> 或 static 缓存
```

- 红线：Tick / 循环体里禁止字符串拼接、GetName()、Format、ToString()

算法：别手写循环 —— Algo 家族

项目全库只用 12 处，潜力很大

```
// 手写查找 (BaseUnitActor.cpp:2040) → 一行代替
const auto* Cache = Algo::FindByPredicate(UnitEffectCaches,
    [EffectKind](const auto& C) { return C.EffectID == EffectKind; });

Algo::AnyOf / Algo::AllOf / Algo::CountIf    // 存在 / 全部 / 计数
```

优势

意图一目了然，谓词可复用，避免手写循环的边界 bug。

排序与二分的两条硬规则

- TArray::Sort 是快排、不稳定 —— 并列元素的相对顺序会变；UI 列表、名次并列用 StableSort (项目 76 : 11 处)
- Algo::BinarySearch 前提是数组已排序 —— 未排序就二分，结果未定义

选型决策表（速查）

所有结论一页汇总，建议贴显示器

场景	用这个	不要用
单位列表 · 特效列表 · 武器槽	TArray	TMap
按键取数据（技能表 / 配置表）	TMap	TArray + 线性 Find
Key 是字符串	FName 当 Key	FString 当 Key
去重 / 查重 / 交并差	TSet	TArray + Contains
有序遍历 + 按键查	TSortedMap / 有序数组 + 二分	TMap
只遍历不持有	TArrayView / const TArray&	按值传参
FIFO 队列	TQueue	TArray + RemoveAt(0)
批量删除不保序	RemoveAtSwap / RemoveAll	循环里逐个 RemoveAt
循环里批量 Add	Reserve + Emplace	裸 Add

提醒：元素 < 100 个时，TArray 线性查找往往比哈希快 —— 别为了「高级」上 TMap

写代码前必问的 5 个问题

再复习一遍第一性原理

- ① 数据多大? <100 个元素, TArray 线性查找可能更快
- ② 访问模式? 每帧遍历 → TArray; 按键取 → TMap; 都要 → 双结构
- ③ 顺序重要吗? 有 → 保序操作 (Remove / StableSort) ; 没有 → SwapRemove / 快排
- ④ 在 Tick 里吗? 是 → 禁字符串、线性查找、NewObject、Trace; 先缓存好
- ⑤ Key 是什么类型? FString 想清楚能否换 FName / 整数

三条总纪律

- 先确认容器类型与调用频率, 再谈优化 (SoundList 案例)
- 逻辑层代码自问: 双端跑, 结果一致吗? (顺序 / 排序 / 定点数)
- 先写对, 再优化 —— 优化以 Profiler 数据为依据, 不凭感觉

03

内存与分配

热路径的默认目标：零堆分配

- 内存分配：内联 vs 堆
- UE 的 GC 原理与 UPROPERTY 标注
- Small Vector 与 TInlineAllocator
- for 循环与 Tick 的分配纪律

内存分配：能内联就内联，能复用就复用

堆分配是隐藏的性能敌人

对比项	栈 / 内联 (对象内部)	堆 (FMemory::Malloc)
分配开销	零 (编译期已固定)	分配器查找、锁、可能触发页
缓存	随对象连续存放，友好	地址离散，命中率低
适用场景	数量小 · 生命周期短 · 热路径	数量大 · 生命周期长

关键认知

- 堆分配真正的成本是「频率」： $\text{每帧} \times \text{单位数} \times \text{次数} = \text{分配器锁竞争} + \text{内存碎片}$
- UObject 的 NewObject 还有 GC 跟踪成本，比普通堆分配更贵
- 热路径 (Tick / 高频调用) 的默认目标：零堆分配

UE 的 GC：标记-清除的引用追踪

UObject 不用手动 delete —— 生命周期交给垃圾回收

① 根集合

当前栈上的 UObject、类默认对象（CDO）、全局对象、UPROPERTY 标记引用的对象

② 标记

从根集合出发，沿 UPROPERTY 引用遍历，可达的对象标记为「存活」

③ 清除

不可达（无任何引用链）的对象，在 GC 回合被统一回收销毁

```
// 你「持有」一个 UObject 的唯一方式：让 GC 看得见
UPROPERTY() USoundBase* MySound; // 根集合追踪 → 存活
USoundBase* TmpSound;           // 看不见 → 随时被回收
```

增量式

每帧分配量达到阈值才触发 GC，分摊卡顿，不是一次性全扫。

- 结论：UObject 不手动 delete；UPROPERTY() = 向 GC 声明「我持有它」
- 项目对照：BattleCore 纯 C++（无 GC，自己管内存）；表现层 UObject 全走 GC

UPROPERTY 标注：与 GC 生死攸关

不标注 = 引擎不认为你持有它 = 随时被回收

```
// ❌ 未标注：GC 扫描不到这个引用，对象随时被回收 → 悬垂崩溃
USoundBase* MySound;

// ✅ 标注：GC 保持它存活
UPROPERTY() USoundBase* MySound;

// 弱引用：不阻止回收，销毁后自动失效
TWeakObjectPtr<AActor> Target;    // IsValid() 判空
```

核心认知

UE 的 GC 是标记-清除：只有被 UPROPERTY 标记的引用才算「持有」。

未标记的 UObject 成员指针 = 「随机崩溃」最常见的来源之一，是正确性问题不是风格问题。

- 规则：持有 UObject 指针的成员一律 UPROPERTY(); 局部临时指针不用标
- 项目对照：BattleCore 纯 C++ 无 UObject，不受影响；表现层所有 Actor / 组件 / 资源引用都要标

Small Vector: 数据量小就不碰堆

SSO —— Small Size Optimization, 小对象优化

原理：对象内部嵌入一小块固定缓冲。

元素放得下 → 直接用内部缓冲，零堆分配；放不下 → 才转到堆上。

表现：对象的大小「变大了一点」，但堆分配「少了很多」。

内部缓冲 [A | B | C | _ | _] ← 第 4 个元素时，转堆

何时用

- 元素数量小且上限可预估（一般 ≤ 32 ）—— 组员列表、技能参数、临时集合
- 热路径中高频创建销毁的容器
- 数量可能很大、不确定 → 别用，老老实实 TArray
- 同一思想：C++ 的 `std::inplace_vector` / `std::string` 的短字符串优化（SSO）

UE 实践: TInlineAllocator 与 TStaticArray

UE 里的小对象优化, 一行模板参数

```
// 前 32 个元素内联在对象里, 超出才转堆
TArray<FUnit*, TInlineAllocator<32>> Members;

// 编译期定长, 完全零堆分配
TStaticArray<int32, 8> GridOffsets;

// 内联字符串缓冲, 替代反复 FString 拼接
TStringBuilder<256> Builder;
Builder.AppendInt(Index);
```

代价:

每个对象多占 $N \times \text{sizeof}(T)$ 字节 —— 别给大量对象 (比如每个单位) 配大容量内联缓冲。

适合放: 小组员列表、技能参数、临时集合。

与 TArray 的对比

- 默认 TArray = 堆分配 (FHeapAllocator) : 对象 16 字节, 但每次操作都走堆
- TInlineAllocator<N> = 内联优先: 对象变大, 但小数据量阶段零堆分配

for 循环内不要申请容器

每次迭代都在构造 + 堆分配 + 析构

```
// ❌ 每次迭代：构造容器 + 可能堆分配 + 析构
for (auto& Group : AllGroups)
{
    TArray<FUnit*> Local;           // 每轮都重新分配
    Local.Add(Group->GetLeader());
}

// ✅ 提到循环外，Reset 复用（容量保留）
TArray<FUnit*> Local; Local.Reserve(32);
for (auto& Group : AllGroups)
{
    Local.Reset();                 // 只清元素，容量还在
    Local.Add(Group->GetLeader());
}
```

关键区别

Reset() 清元素、保留容量；Empty() 清元素、释放容量（下一轮还要重新分配）。

嵌套循环同理：容器能提多外，就提多外。

- 配合 TInlineAllocator：循环内临时小容器，直接零分配

Tick 里不要频繁堆分配

热路径的堆分配会被「单位数 × 帧率」放大

```
                                // ✖ Tick 里的堆分配三连
void Tick(float Dt)
{
    auto* Flash = NewObject<UFlash>(); // 每帧新建 + GC 跟踪
    TArray<FVector> Points;           // 每帧扩容
    FString Msg = FString::Printf(...); // 每帧堆分配
}
```

对策

预创建 + 对象池：Tick 里取/还，不新建

成员变量容器 + Reset 复用

TStringBuilder / 缓存文本，替代拼接

- 项目背景：单位多时，ABaseUnitActor 每帧 Tick × 单位数量，任何一次堆分配都被放大 N 倍
- 纪律：写 Tick 函数时，保证其中「堆分配次数 = 0」

04

类型与生命周期

类型混用与生命周期，是「随机崩溃」的两大来源

- 有符号 / 无符号：std::cmp 安全比较
- 迭代器失效 · 结构体初始化
- 仿函数与 Lambda · UE 委托 · 蓝图纪律
- auto 的拷贝陷阱 · 指针类型选型

int 与 size_t: 隐式转换的陷阱

有符号与无符号混合比较 / 运算，int 会先被转成无符号

```
// ❌ 比较：-1 被转成 4294967295，条件意外成立
int32 Index = -1;
if (Index < V.size())           // 负数 < 无符号 → 变成巨大正数
    Use(V[Index]);              // 越界

// ❌ 减法下溢：size_t 无负数 · 1 - 2 = 巨大数
size_t Diff = A - B;            // 下溢，不是 -1

// ❌ 空容器 + size()-1：同样下溢
for (size_t i = V.size() - 1; ...)
```

为什么会这样

C++ 的隐式转换规则：有符号与无符号混合时，有符号值先转为无符号——负数变成巨大的正数。

结果不是崩溃提示，而是「悄悄错」的越界 / 死循环。

项目实测

- 全库 532 处 size_t、1493 处 std::vector —— 主要在编辑器 / 桥接 / 工具层，与 int 交界处就是雷区
- 注意：UE 容器 TArray::Num() 返回 int32（不是 size_t），UE 内部混算风险低；std 容器 .size() 才是 size_t

安全比较：std::cmp 家族

C++20 起标准库提供；本项目是 C++17，需自实现或升级

```
#include <utility>

// C++20：类型无关的安全比较
std::cmp_less(Index, V.size());           // 有符号/无符号都安全
std::cmp_equal(A, B);
std::cmp_greater(Count, Limit);
std::in_range<int32>(V.size());           // size_t 能否安全放进 int32

// 项目是 C++17 (AutoRTS.Build.cs) → 需自实现
// 逻辑：同符号直接比；异符号先判负数，再转无符号比
```

```
// C++17 简版实现 (可放公共头)
template <typename T, typename U>
bool cmp_less(T a, U b)
{
    if constexpr (std::is_signed_v<T> == std::is_signed_v<U>)
        return a < b;
    else if constexpr (std::is_signed_v<T>)
        return a < 0 || (std::make_unsigned_t<T>)a < b;
    else
        return b >= 0 && a < (std::make_unsigned_t<U>)b;
}
```

排查技巧：编译时开启 -Wsign-compare / -Wsign-conversion (MSVC /W4)，编译器会直接标出这些位置

使用规则

UE 内部：统一用 int32 与 TArray::Num() 比较，天然安全

与 std 容器 / 第三方接口交界处：用 cmp 系列

减法前先判断大小，避免下溢

迭代器失效：循环中删除的危险

删除元素会让迭代器 / 索引失效

```
// ❌ 删除后 It 悬垂，++It 是未定义行为
for (auto It = V.begin(); It != V.end(); ++It)
    if (*It == X) V.erase(It); // 悬垂！

// ✅ std 容器：用 erase 的返回值接住下一个迭代器
for (auto It = V.begin(); It != V.end(); )
    if (*It == X) It = V.erase(It);
    else ++It;

// ✅ UE 容器：RemoveAll / 倒序遍历，不手写迭代器循环
```

核心认知

任何「删除」都可能让正在用的迭代器 / 索引失效——包括 TArray::RemoveAt 和 TMap 的 Remove。

UE 惯例：删除交给容器自身 (RemoveAll / RemoveAtSwap)，不要在循环里自己维护迭代器。

- 与「遍历中删除」页呼应：先删后用的后果，比性能问题更严重（未定义行为）

结构体字段初始化：垃圾值的来源

C++ 内置类型不初始化 = 栈上随机垃圾值

```
// ❌ 三个字段全是随机值
struct FWeaponSlot
{
    int32 Damage;           // 随机！
    float Cooldown;         // 随机！
    UPROPERTY() USoundBase* Sound; // 野指针！GC 会崩
};

// ✅ 类内默认值，一劳永逸
struct FWeaponSlot
{
    int32 Damage = 0;
    float Cooldown = 1.0f;
    UPROPERTY() USoundBase* Sound = nullptr;
};
```

重点

UPROPERTY 的 UObject 指针不置空 → 野指针，GC / IsValid 直接崩，是「随机崩溃」高发源。

UE 惯例：USTRUCT / UCLASS 成员一律给类内默认值，或构造函数显式初始化。

- 结构体新增字段时顺手给默认值 —— 成本一行，收益是永远不用猜

仿函数与 Lambda 的底层原理

Lambda = 编译器生成的匿名仿函数：捕获变成员，调用变 operator()

```
// 仿函数：重载 operator() 的类，可像函数调用，还能携带状态
struct FAdder
{
    float Offset; // 状态
    float operator()(float V) const { return V + Offset; }
};
FAdder Add{ 10.f }; Add(5.f); // → 15

// Lambda 是它的语法糖：编译器生成匿名仿函数类（闭包）
auto AddL = [Offset = 10.f](float V) { return V + Offset; };
// 展开 ≈ struct Closure { float Offset; float operator()(float) const; };
```

三个推论

捕获 = 成员变量：生命周期与 lambda 对象绑定

[&Speed] 存的是指向栈对象的引用 → 回调执行时已悬垂（下页详解）

lambda 大小 = 捕获之和；空捕获 ≈ 函数指针，零开销

类型匿名 → 只能用 auto / 模板接收

与 UE 委托的关系

CreateLambda = 在委托里保存一个仿函数对象；CreateUObject = 保存弱引用 + 成员函数指针 —— 都是「状态 + 调用」的封装，下一页委托页展开。

理解底层之后：写 Lambda 前先想「它会把什么带进闭包、活多久」

Lambda 的坑：捕获与生命周期

延迟 / 异步回调里，引用和裸指针都可能悬垂

```
float Speed = 10.f;
// ✗ 按引用捕获局部变量：回调执行时变量已销毁
SetTimer(1s, [&Speed] { Move(Speed); }); // 悬垂引用

// ✗ 捕获 this: this 先销毁 → 悬垂调用
Button->OnClicked.AddLambda([this] { DoThing(); });

// ✓ 按值捕获：拷贝快照，无生命周期包袱
SetTimer(1s, [Speed] { Move(Speed); });
// ✓ UE 委托绑定成员函数：销毁自动解绑
FTimerDelegate::CreateUObject(this, &UMyActor::OnTimeout);
```

提问清单

这个 Lambda 什么时候执行？对象会不会先死？

捕获的对象是「活的」还是「快照」？

无法保证存活 → UE 委托自动解绑（`CreateUObject`）。

- 默认按值捕获；引用捕获只用于「同栈帧内必然先执行」的场景

表现层优先用 UE 委托：自动维护生命周期

AddUObject / CreateUObject：对象销毁自动解绑

```
// ✅ 对象销毁时委托系统自动解绑，不会悬垂调用
OnUnitDied.AddUObject(Label, &UUnitLabel::OnDied);

// ✅ 动态委托：蓝图可见，可跨蓝图绑定
DECLARE_DYNAMIC_MULTICAST_DELEGATE_OneParam(FOnHealthChanged, float, NewHealth);
MyHealthChanged.AddDynamic(this, &UHealthBar::OnHealthChanged);

// ✅ 回调风格：创建一次性回调，绑定对象销毁自动失效
FTimerHandle H;
GetWorld()->GetTimerManager().SetTimer(H, FTimerDelegate::CreateUObject(this,
&UMyActor::OnTimeout), 1.f, false);
```

为什么

委托内部持有 TWeakObjectPtr 弱引用：绑定对象被 GC 回收，委托自动失效，绝不悬垂。

裸函数指针 / 裸 Lambda 捕获 this：忘了手动解绑就悬垂。

- 适用：表现层事件（单位阵亡、状态变化、UI 刷新、计时器回调）

蓝图：构造函数与 Tick 的纪律

构造阶段不是运行时；脚本 Tick 每帧都很贵

构造函数（Construction Script）

只做「搭建」：组件、默认值、外观。拿 World / Player / 运行时数据放在 BeginPlay。
构造阶段对象还没进世界，很多查询为空，别在这里写业务逻辑。

Tick（Event Tick）

蓝图 Tick 跑在脚本虚拟机里，每帧解释执行 —— 单位一多开销被放大。
能事件驱动就不进 Tick；必须每帧的活尽量下沉到 C++，蓝图只挂表现。

- 项目架构对照：BattleCore 逻辑层根本没有蓝图；表现层也优先 C++ Tick + 蓝图事件
- 蓝图 Tick 里禁：每帧查找、每帧创建、每帧字符串拼接

auto 的拷贝陷阱

auto 默认推导成「值」——大对象会被整个拷贝

```
// ❌ 每个元素拷贝一次 ( TArray / FString / 大结构体很贵 )  
for (auto Elem : BigArray)  
    Use(Elem);           // Elem 是副本  
  
// ✅ 遍历只读 : const auto&, 零拷贝  
for (const auto& Elem : BigArray)  
    Use(Elem);  
  
// ✅ 要修改 : auto& ; 要副本才用裸 auto  
for (auto& Slot : WeaponSlots)           // 项目现有写法正确
```

注意

元素是 UObject 指针时 auto 无成本 (拷指针), 但 TArray、FString、FTransform 这类值类型就是深拷贝。

范围 for 里: 默认 const auto&, 除非真要改。

- auto 的另一面: 类型隐晦。团队惯例: 能写具体类型 (int32 / FUnit*) 就写, 只在长类型名时用 auto

指针类型选型：一份生命周期契约

每种类型都声明了「谁持有、谁回收」—— 选错就是悬垂与崩溃

类型	生命周期	适用场景
普通值 (int32 / FVector / TArray)	自身管理 · 无问题	数据成员、临时变量 —— 默认首选
UObject* + UPROPERTY()	GC 强引用 · 保活	默认选择：资源 / 组件 / Actor 引用
TObjectPtr<UObject>	GC 强引用 · 支持延迟加载	替代 UObject 裸指针 (4.27 可用)
TWeakObjectPtr<UObject>	弱引用 · 不阻止回收 · 自动失效	缓存 / 观察者 (对象可能销毁)
TSharedPtr / TWeakPtr	引用计数 (非 UObject)	纯 C++ 对象共享所有权
裸指针 (纯 C++)	手动管理	BattleCore 逻辑层：谁创建谁负责

```
// 选型三问：谁持有？会销毁吗？GC 管得到吗？
UPROPERTY() USoundBase* MySound;    // 默认：GC 强引用保活
TWeakObjectPtr<AActor> Target;      // 弱引用：销毁后自动判空
TSharedPtr<FConfig> Cfg;             // 非 UObject：引用计数
```

怎么选

默认：普通值 / UPROPERTY 指针

要安全地观察可能销毁的对象 → TWeakObjectPtr

逻辑层纯 C++ → 裸指针 + 所有权约定

- 项目对照：BattleCore 无 GC 用裸指针；表现层全部 UObject 体系，引用一律 UPROPERTY

05

接口与协作

接口是给队友读的，让他们少猜

- 返回值设计：最小接口 · TOptional
- 返回引用 or 快照拷贝
- 降低合作者的心智负担

接口设计：别把全量容器甩给调用方

返回值只给「对方真正需要的能力」

```
//      内部结构泄漏 + 每次查找都扫一遍全量
//      返回全量数组，调用方再自己查找 —
const TArray<FUnit*>& GetAllUnits() const;
// 调用方：FindUnit(GetAllUnits(), Id) ← 反复线性扫

// ✓ 最小接口：直接给结果，内部随便怎么存
FUnit* FindUnit(int32 Id);          // 找不到返回 nullptr
bool HasUnit(int32 Id) const;
int32 CountUnits() const;
```

为什么

「给你全部，你自己找」= 调用方写重复逻辑 + 你的内部实现被锁死（以后改存储结构会波及所有人）。

数据留在内部，能力通过方法提供 —— 这也是降低心智负担的一部分。

- 例外：遍历本身就是目的（渲染列表、序列化）时，返回 const 引用 + 文档说明

返回值语义：无结果用 TOptional

0 可能是合法值 —— 别用魔法值表达「没有」

```
// ❌ 0 表示「没有」？但座位 0 是合法的！
int32 FindSeat() { return bHasSeat ? SeatIndex : 0; } // 歧义

// ✅ TOptional：明确表达「可能无结果」
TOptional<int32> FindSeat()
{
    return bHasSeat ? TOptional<int32>(SeatIndex) : TOptional<int32>();
}

// 调用方
if (TOptional<int32> Seat = FindSeat())
    Use(Seat.GetValue());
```

适用场景

位置、索引、ID 这类「0 有意义」的返回值

查询类接口：找到/没找到是两种状态，不是一种

UE 用 TOptional<T>（不是 std::optional）

- 同理：-1 表达「没有」也要小心 —— 用 TOptional 一劳永逸

返回引用 or 拷贝：取决于访问者的需求

只要「当时的快照」→ 拷贝；要长期持有 / 修改 → 引用

```
// ✅ 快照语义：返回拷贝，调用方无生命周期包袱
FVector GetPosition() const;           // 值类型，拷贝便宜

// ✅ 引用语义：长期持有 / 修改原数据
const TArray<FUnit*>& GetMembers() const; // 零拷贝

// ❌ 绝不返回「可能已销毁对象」的引用
const FUnit*& GetDeadUnit() const;      // 悬垂风险
```

决策

只读一次 → 拷贝（简单、安全、线程友好）

反复访问 / 要修改 → const 引用

要跨作用域持有 → 拷贝 或 TSharedPtr，别留引用

- 快照语义的代价是一次拷贝 —— 在「正确性」面前，这份拷贝通常很便宜

类与函数设计：降低合作者的心智负担

接口是给队友读的 —— 让他们少猜

命名即文档

FindUnit 就返回单位；别叫 Find 却返回 bool + 出参

单一职责

一个函数只做一件事；太长就拆（文件同理）

参数最少化

能用 1 个参数不用 3 个；给合理的默认值

const 正确性

能 const 就 const —— 接口自己声明「我不改你的数据」

错误显式化

找不到 → 返回 nullptr / TOptional；不要魔法值

判据：你的接口，队友不看实现能正确用吗？不能 → 继续简化

全课总结：结构总览

五个章节，一条主线 —— 先懂架构，再写代码

01 原理与架构

先懂框架：帧同步 × 逻辑/表现分层 —— 所有纪律的根源

02 容器与算法

数据怎么存：内存布局 → 选型 —— 性能的起点

03 内存与分配

内存怎么管：GC 原理 × 零堆分配纪律

04 类型与生命周期

正确性保障：类型安全 × 指针契约 × 回调生命周期

05 接口与协作

协作设计：返回值语义 × 降低心智负担

第一性原理：复杂 = 简单模块的组合；业务本质 = 增删改查 —— 锚定原理，自检每一行代码

谢谢 · Q & A